

■ PYTHON TEXT ADVENTURE GAME

SESSION 1 — DETAILED LESSON NOTES

Subject	Python Programming (Beginners)
Session	1 of Series
Theme	Build a Text Adventure Game ■
Core Concepts	print(), input(), Variables, if/else
Target Audience	School Students — No prior coding experience
Duration	~60 minutes

■ By the end of this session, students will be able to write a fully working interactive text adventure game in Python using print(), input(), variables, and if/else branching.

1. LESSON OVERVIEW

What Will Students Build?

Students will construct a fully interactive **Jungle Adventure Game** in Python from scratch. The game greets the player, asks for their name, presents two paths (LEFT or RIGHT), and narrates a different outcome depending on their choice — all using just the four core Python concepts taught in this session.

Cultural Connection

The session is inspired by **Amar Chitra Katha** choose-your-own-path comics — a familiar and beloved format for many students. Just like those stories let readers decide what happens next, students will write code that branches based on player decisions. This analogy makes the abstract concept of if/else immediately tangible.

Learning Objectives

#	Objective	Bloom's Level
1	Explain what print() does and use it to display messages	Knowledge / Apply
2	Explain what input() does and use it to capture player responses	Knowledge / Apply
3	Create and use variables to store and recall player data	Apply
4	Write if/else statements that branch a story based on player choice	Apply / Analyse
5	Combine all four concepts to build a complete mini game	Synthesise

Materials Needed

- A computer with Python 3.x installed (or an online IDE like replit.com / thonny)
- A text editor or Python IDLE
- This printed lesson guide (optional)
- Whiteboard or projector for live demonstrations

2. CORE CONCEPTS EXPLAINED

2.1 The print() Function — The Game's Voice

The **print()** function sends text to the screen. Think of it as the narrator of your story — every message the player reads is delivered by print().

■ **Analogy:** print() is like a Doordarshan newsreader — it announces whatever you tell it to say, clearly and immediately.

Syntax:

```
1 print("Your message here")
```

Examples used in the game:

```
1 print("Welcome to the Jungle Adventure!")
2 print("You stand at a fork in the path.")
3 print("LEFT: Enter the mysterious cave")
4 print("RIGHT: Follow the sparkling river")
```

Each print() call shows one line of text. Multiple print() calls appear one after another, building up the story like lines in a comic.

2.2 The input() Function — The Game Listens

The **input()** function pauses the program and waits for the player to type something. When the player presses Enter, input() returns whatever they typed.

■ **Analogy:** input() is like a teacher writing a question on the blackboard — the class (player) must answer before the lesson (game) continues.

Syntax:

```
1 answer = input("Your question here: ")
```

Examples used in the game:

```
1 name = input("What is your name? ")
2 choice = input("Which path do you choose? LEFT or RIGHT: ")
```

Key Point: print() vs input()

- **print()** — one-way: the computer talks to the player.
 - **input()** — two-way: the player talks back to the computer.
- Together they create a real conversation, turning code into a game.

2.3 Variables — Magic Dabba Boxes

A **variable** is a named container that stores a piece of information. Once you put something in a variable you can use it again anywhere in your program.

■ **Analogy: Variables are like steel tiffin dabba boxes — each one has a name written on the lid and stores something delicious (or useful!) inside.**

```
1 # Storing the player's name
2 name = input("What is your name? ")
3
4 # Using the variable later
5 print("Hello, " + name + "! Welcome to the jungle.")
6 print("Good luck, " + name + "!")
```

Rule	Example	Explanation
Assignment	name = "Priya"	Store the value 'Priya' in the box called name
Recall	print(name)	Fetch whatever is stored in name and print it
Concatenation	"Hello, " + name	Join text strings together
Re-use	print(name) ... print(name)	Use the same variable as many times as you like

2.4 if/else Statements — The Railway Signal

An **if/else** statement lets the program choose between two different blocks of code depending on whether a condition is True or False. This is what makes a game feel interactive — choices actually matter!

■ **Analogy: if/else is like a railway signal — green sends the train one way, red sends it another. Your code reads the player's choice and 'switches the track'.**

Syntax:

```
1 if condition:
2     # code that runs when condition is TRUE
3 else:
4     # code that runs when condition is FALSE
```

Game example:

```
1 if choice == "LEFT":
2     print("You enter the cave and find glittering treasure!")
3     print("Well done, " + name + "!")
4 else:
5     print("You follow the river and meet friendly dolphins!")
6     print("What a magical journey, " + name + "!")
```

Syntax Rules to Remember

1. The condition comes directly after **if** with no parentheses needed.
2. The line ends with a **colon (:)** — this is compulsory.
3. The code inside each block must be **indented** (4 spaces or 1 Tab).
4. Use **==** (double equals) to compare values, not **=** (single equals).

3. BUILDING THE GAME — STEP BY STEP

Below is a guided walkthrough of how to build the complete Jungle Adventure Game. Each step introduces one new concept. Encourage students to type — not copy-paste!

STEP 1 Welcome Messages

Use `print()` multiple times to build a title screen. Notice how each line appears on its own row.

```
1 print("=====")
2 print("  WELCOME TO THE JUNGLE ADVENTURE!")
3 print("=====")
```

STEP 2 Ask the Player's Name

`input()` pauses the game and waits. The player's answer is stored in the variable `name` for use throughout the game.

```
1 name = input("What is your name, brave explorer? ")
```

STEP 3 Personalised Greeting

We concatenate (join) strings with `+` to include the player's name, making the experience feel personal.

```
1 print("Great to meet you, " + name + "!")
2 print("Your jungle adventure begins now...")
```

STEP 4 Present the Fork in the Path

Use `print()` to describe the scene and list the available choices clearly.

```
1 print("")
2 print("You reach a fork in the jungle path.")
3 print("LEFT: Enter the mysterious cave")
4 print("RIGHT: Follow the sparkling river")
```

STEP 5 Capture the Player's Choice

`input()` waits for the player to type. We use `.upper()` so 'left' and 'LEFT' are treated the same way.

```
1 choice = input("Which path do you choose? (LEFT/RIGHT): ")
2 choice = choice.upper()    # accept lowercase too
```

STEP 6 Branch with if/else

The if/else block reads the player's choice and tells a different part of the story. This is the heart of any interactive game.

```
1 if choice == "LEFT":
2     print("You enter the cave...")
3     print("You find a glittering chest of treasure!")
4     print("Congratulations, " + name + "! You win!")
5 else:
6     print("You follow the river...")
7     print("You meet friendly dolphins who guide you home!")
8     print("What a journey, " + name + "!")
```

Complete Game — Full Code Listing

Here is the entire game in one place. Students should aim to type this from memory using their step-by-step notes before checking here.

[illegible]

4. QUIZ — CHECK YOUR UNDERSTANDING

Answer these questions to check what you have learned. Correct answers are highlighted in green.

Q1. Which function displays text on the screen?

- A) input()
- ✓ B) print()
- C) variable()
- D) display()

Tip: The print newsreader reads out your message!

Q2. What does input() do?

- A) Displays text on screen
- ✓ B) Waits for the player to type something
- C) Closes the program
- D) Creates a new file

Tip: Like a teacher asking a question — the class must answer.

Q3. Complete the if statement: if choice == 'LEFT' ____

- A) semicolon ;
- ✓ B) colon :
- C) full stop .
- D) comma ,

Tip: Python uses a colon to start every if, else, elif, for, while, and def block.

Q4. What symbol is used to compare two values in an if statement?

- A) = (single equals)
- ✓ B) == (double equals)
- C) >= (greater or equal)
- D) != (not equal)

Tip: Single = assigns a value; double == checks if two values are the same.

Q5. Which of the following correctly stores a player's name?

- A) print("name")
- ✓ B) name = input("Your name: ")
- C) input(name)
- D) variable name = "player"

Tip: We use variable = input(...) to capture whatever the player types.

5. ACTIVITIES & EXTENSION CHALLENGES

Core Activity — Run and Modify the Game

Run the complete game at least twice — once choosing LEFT, once choosing RIGHT. Then change the setting to one of the following and rewrite both story paths:

- ■ Cricket ground
- ■ School
- ■ Market
- ■ Outer Space
- ■■ Mountain peak
- ■ Ocean floor

Bonus Challenge — Add a Third Path with elif

■ elif (else-if) lets you add a third (or fourth, or fifth...) branch. Example: What if the player types "CLIMB" to go up a tree?

```
1 if choice == "LEFT":
2     print("Cave path story...")
3 elif choice == "CLIMB":
4     print("You climb a tall tree and spot a hidden village!")
5 else:
6     print("River path story...")
```

Extension Ideas for Advanced Students

Ask two questions in a row: Use a second input() after the first choice to create a deeper story.

Add a score: Create a variable score = 0 and increase it with score = score + 10 when the player makes a good choice.

Use a loop: Wrap the game in a while loop so players can replay without restarting the program (preview of Session 2).

6. TEACHER / FACILITATOR NOTES

Common Mistakes to Watch For

■ Missing colon after if/else

if choice == "LEFT" ← SyntaxError; add the colon!

■ Single = instead of ==

if choice = "LEFT" ← use == for comparison

■ Missing indentation

Code inside if/else must be indented 4 spaces or 1 Tab

■ Quotation mismatch

Mixing " and ' in one string: print("Hello") ← Error

■ Variable used before assignment

Make sure name = input(...) runs before any print(name)

Suggested Lesson Timing

Time	Activity
0 – 5 min	Welcome, icebreaker — what's your favourite game?
5 – 10 min	Show the finished game running; build excitement
10 – 20 min	Teach print() with live demo; students try it
20 – 30 min	Teach input() + variables; build steps 1-3 together
30 – 45 min	Teach if/else; build steps 4-6 together
45 – 55 min	Students run, test, and customise their game
55 – 60 min	Quiz review + preview of Session 2 (loops!)

7. PREVIEW — SESSION 2

Coming Up in Session 2

In Session 2 we will make the game even more powerful by introducing loops and more complex decision trees:

- while loops — let the game run again and again
- elif chains — add three, four, even five story branches

- Nested if/else — choices within choices for richer stories
- A scoring system — reward the player for smart decisions

■ Remember: Every great game designer started exactly where you are today. Keep coding, keep exploring, and keep building amazing stories! ■